



+94 771 051 138

229/1 Bernard Soysa Mawatha, Colombo 05 swivelgroup.com.au

Practical Assignment

Overview

The **Event Planner App** is a Flutter application designed to showcase a candidate's skills in mobile app development, including user authentication, push notifications, and Firebase integration. The app should demonstrate proficiency in Flutter, best practices, appropriate architecture patterns, UI design, state management, and external API integration. Use the **wireframes** as a guide to implement the UI layout as closely as possible.

Wireframes

Refer to the Figma design for detailed visual structure:

Wireframes

- **Login Screen:** The **Login Screen** allows users to sign in using their email and password.
- **Sign Up Screen:** The **Sign-Up** page allows new users to create an account by providing their details.
- **First-Time Login:** If the user is logging in for the first time, they will be directed to a profile setup screen where they can complete their account details.
 - **Upload Profile Picture:** Users are prompted to upload a profile picture either by choosing from their gallery or taking a new photo.
 - **Personal Information**
 - ♣ First Name
 - ♣ Last Name
 - ♣ Email (pre-filled based on signup)
 - ♣ Phone Number
 - ♣ Mailing Address
- **Home Page:** After login, display a home page with the following features

- **Top Image Slider:** Use the first 10 images from the following API to create a slider
 - ♣ API: <https://jsonplaceholder.typicode.com/photos>
- **Event Name:** Display the current event's name (choose any name).
- **Event Organizers List:** Show a list of event organizers using the following API.
 - ♣ API: <https://jsonplaceholder.typicode.com/users>
- **Horizontal Image List:** Implement a horizontal list with descriptions, showing only the first 10 images.
 - ♣ API: <https://jsonplaceholder.typicode.com/photos>
- **Post Count and Navigation:** Display the total number of posts. Clicking on it should navigate to a page listing the posts, along with comments. Design the post and comments list according to your preference.
 - ♣ Posts API: <https://jsonplaceholder.typicode.com/posts>
 - ♣ Comments API: <https://jsonplaceholder.typicode.com/comments>
- **User Profile:** Implement a tab-based navigation with a **Profile** tab that allows users to view and edit their profile information.
- **Drawer Navigation:** add a drawer that includes a **Logout** button and application version in the bottom.

Implementation Guidelines

1. Project Structure and Code Readability
 - **Folder Structure:** Organize your project so that it adheres to the best modern practices for file and folder organization
 - **Readability:** Ensure the code is clean, well-commented, and uses meaningful variable and function names.
2. Frontend implementation
 - Use **Flutter Widgets**.
 - Use **provider**, **riverpod**, or **bloc** for state management.
 - **User Authentication:** Implement user signup and login functionalities using **Firebase Authentication**. Which will allow users to create an account, log in, and log out.
 - **Firestore Integration:** Use Firestore for user account management, including profile data and profile images. Configure push notifications using **Firestore Cloud Messaging (FCM)**
 - Schedule three push notifications to be sent to all users daily at 8 am, 12 noon, and 5 pm. The push notification messages can be anything

- **Form Handling:** Implement proper form handling for the **Login**, **Signup** and **Edit Profile** forms. Each form field should be validated in real time as per the specified rules:
 - Password: Must contain
 - ☐ At least **1 capital letter** (A-Z)
 - ☐ At least **1 lowercase letter** (a-z)
 - ☐ At least **1 number** (0-9)
 - ☐ Minimum 3 **characters**
 - Email: Standard email validation (e.g., using regex).
 - First Name and Last Name: Alphabetic characters only, with a minimum of 3 and a maximum of 10 characters.
 - Phone Number: Validate the format for Sri Lankan numbers.
 - Address: Minimum of 6 and a maximum of 50 characters.
 - Ensure that the form handles validation correctly and highlights errors by adding a red border around the incorrect field and showing a helpful error message beneath it.
- **Unit Test:** Use **flutter_test** or any testing framework to write unit tests. Focus on writing unit tests for critical components. Ensure that form validation works as expected and that the API calls are triggered correctly on form submission. Mock API responses where necessary.
- **Log Handling:** Implement clear logging mechanisms, especially for key operations. Avoid logging sensitive information.
- **Exception Handling:** Implement exception handling with best practices to deal with potential errors. Focus on handling exceptions for Network Requests, Database Operations, Asynchronous Operations, User Input Validation, Navigation and Routing and other key areas.

Submission Guidelines

- **GitHub Repository:** Upload your project to GitHub and share the repository link. The repository should include:
 - A clear README file with instructions for setting up and running the project.
 - Screenshots or descriptions of any non-standard configurations.
- **Demo Video:** Provide a **demo video** (no voice required) showcasing the app's functionality.
- **README:** The README should include:
 - A brief description of the project and the technologies used.
 - Instructions for cloning the repository and running the application.
 - Any known issues or limitations of the project.

- **Firestore integration:** Include screenshots or documentation of Firestore integration.
- **Note:** Make sure your submission is fully functional, and all necessary dependencies are included. The application should run without errors, and at least 80% of the core functionality should be implemented to pass the assessment.

Assessment Expectations

We will evaluate your submission based on the following criteria:

1. **Ease of Setup:**
 - Ensure the code is easy to set up and run. Include detailed instructions in the README file.
2. **Code Quality:**
 - The code must be organized and easy to read.
 - Your components should be reusable and follow modern best practices.
 - We expect **clean architecture** principles, including separation of concerns and proper state management.
 - Using **Typescript** to type check the code. (optional for interns and associate engineers)
3. **Functionality:**
 - All the required features must be implemented and functional. At least 80% of the functionality is expected.
4. **Error Handling:**
 - Proper error handling for edge cases and API failures.
5. **Logs:**
 - Implement clear logging mechanisms, especially for key operations. Avoid logging sensitive information.
6. **Testing:**
 - Provide **unit tests** only for key features
 - Unit testing is mandatory for senior-level submissions but optional for interns and associate engineers.

Rejection Criteria

Your submission may be rejected if it:

- Lacks clear setup instructions or a proper README file.
- Does not compile or run due to missing dependencies or broken code.

- Fails to implement at least 80% of the required functionality.
- Has disorganized code with poor readability or structure.



Happy Coding!!!